

Natural Computing

LMU Munich
summer term 2025

Thomas Gabor



Can cellular automata
explain the universe?

Consider the following variation of a 1D cellular automaton (cf. Definitions 1 and 2 on the definition sheet): Let the state space $\mathcal{X} = (V \rightarrow \mathbb{Z} \times \mathbb{Z})$. Let

$$f((v_l, p_l), (v_c, p_c), (v_r, p_r)) = (v_l^+ + v_r^- + \lfloor \frac{(p_l - p_c)^+}{2} \rfloor - \lfloor \frac{(p_r - p_c)^+}{2} \rfloor, \\ p_c - |v_c| + v_l^+ - v_r^- + \lfloor \frac{(p_l - p_c)^+}{2} \rfloor + \lfloor \frac{(p_r - p_c)^+}{2} \rfloor - \lfloor \frac{(p_c - p_l)^+}{2} \rfloor - \lfloor \frac{(p_c - p_r)^+}{2} \rfloor)$$

where $x^+ = \begin{cases} x & \text{if } x > 0, \\ 0 & \text{otherwise,} \end{cases}$ and $x^- = \begin{cases} x & \text{if } x < 0, \\ 0 & \text{otherwise,} \end{cases}$ for any $x \in \mathbb{Z}$.

Note: $\lfloor x \rfloor$ for $x \in \mathbb{R}$ is x rounded down to an integer as implemented in a typical `floor` function (see Python, e.g.).

From the start configuration of this 1d cellular automaton with 5 cells below, compute the next 3 time steps.

(0, 0)	(+1, +1)	(0, 0)	(-1, +1)	(0, 0)

Example 1 (Cellular Automaton for 1D Particle Collision, inspired by [1]). Consider the following variation of a 1D cellular automaton (cf. Definitions 1 and 2 on the definition sheet): Let the state space $\mathcal{X} = (V \rightarrow \mathbb{Z} \times \mathbb{Z})$. Let

$$f((v_l, p_l), (v_c, p_c), (v_r, p_r)) = (v_l^+ + v_r^- + \lfloor \frac{(p_l - p_c)^+}{2} \rfloor - \lfloor \frac{(p_r - p_c)^+}{2} \rfloor, \\ p_c - |v_c| + v_l^+ - v_r^- + \lfloor \frac{(p_l - p_c)^+}{2} \rfloor + \lfloor \frac{(p_r - p_c)^+}{2} \rfloor - \lfloor \frac{(p_c - p_l)^+}{2} \rfloor - \lfloor \frac{(p_c - p_r)^+}{2} \rfloor)$$

where $x^+ = \begin{cases} x & \text{if } x > 0, \\ 0 & \text{otherwise,} \end{cases}$ and $x^- = \begin{cases} x & \text{if } x < 0, \\ 0 & \text{otherwise,} \end{cases}$ for any $x \in \mathbb{Z}$.

Note: $\lfloor x \rfloor$ for $x \in \mathbb{R}$ is x rounded down to an integer as implemented in a typical `floor` function (see Python, e.g.). From the start configuration of this 1d cellular automaton with 5 cells below, compute the next 3 time steps.

(0, 0)	(+1, +1)	(0, 0)	(−1, +1)	(0, 0)

Example 1 (Cellular Automaton for 1D Particle Collision, inspired by [1]). Consider the following variation of a 1D cellular automaton (cf. Definitions 1 and 2 on the definition sheet): Let the state space $\mathcal{X} = (V \rightarrow \mathbb{Z} \times \mathbb{Z})$. Let

$$f((v_l, p_l), (v_c, p_c), (v_r, p_r)) = (v_l^+ + v_r^- + \lfloor \frac{(p_l - p_c)^+}{2} \rfloor - \lfloor \frac{(p_r - p_c)^+}{2} \rfloor, \\ p_c - |v_c| + v_l^+ - v_r^- + \lfloor \frac{(p_l - p_c)^+}{2} \rfloor + \lfloor \frac{(p_r - p_c)^+}{2} \rfloor - \lfloor \frac{(p_c - p_l)^+}{2} \rfloor - \lfloor \frac{(p_c - p_r)^+}{2} \rfloor)$$

$$\text{where } x^+ = \begin{cases} x & \text{if } x > 0, \\ 0 & \text{otherwise,} \end{cases} \text{ and } x^- = \begin{cases} x & \text{if } x < 0, \\ 0 & \text{otherwise,} \end{cases} \text{ for any } x \in \mathbb{Z}.$$

Note: $\lfloor x \rfloor$ for $x \in \mathbb{R}$ is x rounded down to an integer as implemented in a typical `floor` function (see Python, e.g.).
 From the start configuration of this 1d cellular automaton with 5 cells below, compute the next 3 time steps.

(0, 0)	(+1, +1)	(0, 0)	(−1, +1)	(0, 0)
(0, 0)	(0, 0)	(0, 2)	(0, 0)	(0, 0)
(0, 0)	(−1, +1)	(0, 0)	(+1, +1)	(0, 0)
(−1, +1)	(0, 0)	(0, 0)	(0, 0)	(+1, +1)

```

1 from math import floor
2
3 def eplus(x):
4     return x if x > 0 else 0
5
6 def eminus(x):
7     return x if x < 0 else 0
8
9 def f(left,center,right):
10     vl,pl = left
11     vc,pc = center
12     vr,pr = right
13     pressure_from_left = floor(eplus(pl-pc)/2)
14     pressure_from_right = floor(eplus(pr-pc)/2)
15     pressure_towards_left = floor(eplus(pc-pl)/2)
16     pressure_towards_right = floor(eplus(pc-pr)/2)
17     vnew = eplus(vl) + eminus(vr) + pressure_from_left - pressure_from_right
18     pnew = pc - abs(vc) + eplus(vl) - eminus(vr) + pressure_from_left + pressure_from_right - pressure_towards_left - pressure_towards_right
19     return vnew, pnew
20
21 def apply(vector, f):
22     new_vector = []
23     for e,element in enumerate(vector):
24         new_element = f(vector[e-1], element, vector[(e+1)%len(vector)])
25         new_vector.append(new_element)
26     return new_vector
27
28 rounds = range(4)
29 vector = [(0,0), (+1,+1), (0,0), (-1,+1), (0,0)]
30 for _ in rounds:
31     print(vector)
32     vector = apply(vector,f)

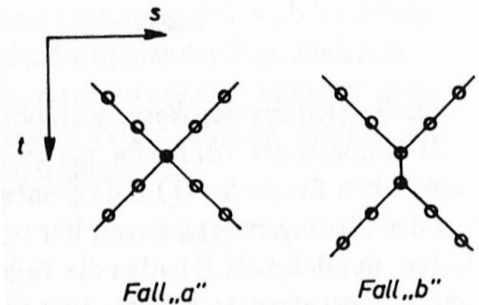
```

state space $\mathcal{X} = (V \rightarrow \mathbb{Z} \times \mathbb{Z})$

t_0	v								
	p					+1	+1		
t_1	v				-1		+1		
	p				+1			+1	
t_2	v			-1				+1	
	p		+1						+1

state space $\mathcal{X} = (V \rightarrow \mathbb{Z} \times \mathbb{Z})$

t_0	v								
	p					+1	+1		
t_1	v				-1			+1	
	p				+1			+1	
t_2	v				-1				+1
	p		+1						+1



Konrad Zuse

1910 – 1995

- built first programmable computer (Zuse Z3)
- designed first high-level programming language (Plankalkül)



source: Wikipedia

What about
non-local
information?

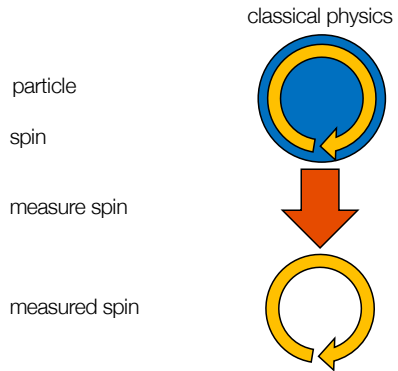
Quantum Computing

What is quantum computing?

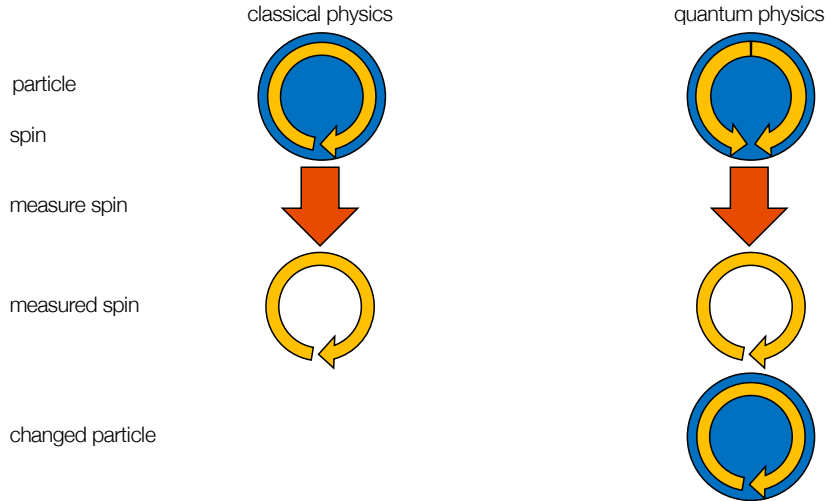
What is quantum computing
within natural computing?

two strange
quantum effects

Superposition



Superposition



Definition 5 (quantum state). Let $\mathcal{X}$ be an arbitrary state space $\mathcal{X}$. Let $\mathcal{Q}(\mathcal{X})$ be the space of superposition states with bases in $\mathcal{X}$, i.e., each $q \in \mathcal{Q}(\mathcal{X})$ has the form

$$q = \sum_{x \in \mathcal{X}} c_x |x\rangle$$

where $c_x \in \mathbb{C}$ for any $x \in \mathcal{X}$ is called an amplitude so that

$$\sum_{x \in \mathcal{X}} |c_x|^2 = 1$$

and $|x\rangle$ (read “ket x ”) denotes a vector corresponding to the state $x \in \mathcal{X}$.

We use the following terminology:

- If for more than one $x \in \mathcal{X}$ it holds that $c_x \neq 0$, then q is called a *superposition* state.

$$q = \sum_{x \in \mathcal{X}} c_x |x\rangle$$

We use the following terminology:

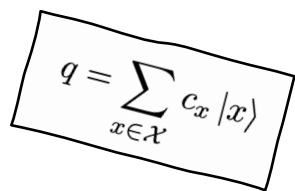
- If for more than one $x \in \mathcal{X}$ it holds that $c_x \neq 0$, then q is called a *superposition* state.
- If $\mathcal{X} = \mathcal{X}_A \times \mathcal{X}_B$, i.e., $\mathcal{X}$ can be constructed as a product of two other state spaces $\mathcal{X}_A, \mathcal{X}_B$, then it follows that q has the form

$$q = \sum_{\substack{x_A \in \mathcal{X}_A, \\ x_B \in \mathcal{X}_B}} c_{x_A, x_B} |x_A\rangle \otimes |x_B\rangle$$

where $\otimes$ is the tensor product. If q cannot be written as

$$q = \left(\sum_{x_A \in \mathcal{X}_A} c_{x_A} |x_A\rangle \right) \otimes \left(\sum_{x_B \in \mathcal{X}_B} c_{x_B} |x_B\rangle \right),$$

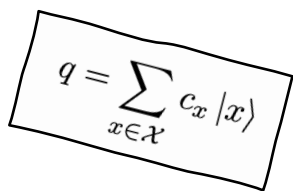
then we call q an *entangled* state.


$$q = \sum_{x \in \mathcal{X}} c_x |x\rangle$$

- If $\mathcal{X} = \{0, 1\}$ (i.e., any $x \in \mathcal{X}$ is a bit), then any $q \in \mathcal{Q}(\mathcal{X})$ is called a *qubit* and can be written as

$$q = c_0 |0\rangle + c_1 |1\rangle$$

for $c_0, c_1 \in \mathbb{C}$ with $|c_0|^2 + |c_1|^2 = 1$.


$$q = \sum_{x \in \mathcal{X}} c_x |x\rangle$$

- If $\mathcal{X} = \{0, 1\}$ (i.e., any $x \in \mathcal{X}$ is a bit), then any $q \in \mathcal{Q}(\mathcal{X})$ is called a *qubit* and can be written as

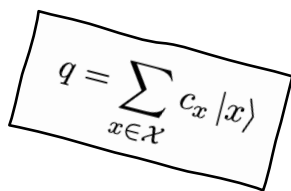
$$q = c_0 |0\rangle + c_1 |1\rangle$$

for $c_0, c_1 \in \mathbb{C}$ with $|c_0|^2 + |c_1|^2 = 1$.

- If $\mathcal{X} = \{0, 1\}^n$ for any $n \in \mathbb{N}$ (i.e., any $x \in \mathcal{X}$ is a bit register), then any $q \in \mathcal{Q}(\mathcal{X})$ is called a *qubit register* of size n and can be written as

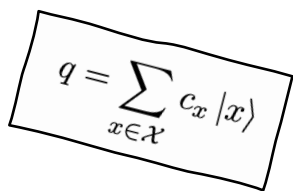
$$q = \sum_{i=0}^{2^n-1} c_i |i\rangle = c_{0\dots 0} |0\dots 0\rangle + \dots + c_{1\dots 1} |1\dots 1\rangle$$

where we usually denote i in binary as illustrated above.


$$q = \sum_{x \in \mathcal{X}} c_x |x\rangle$$

- When we *measure* a quantum state $q \in \mathcal{Q}(\mathcal{X})$, we generate classical state $M(q) \in \mathcal{X}$ so that for all $x \in \mathcal{X}$ it holds that

$$\Pr(M(q) = x) = |c_x|^2.$$


$$q = \sum_{x \in \mathcal{X}} c_x |x\rangle$$

0	0	0	0	0	0	0	0
1	1	1	1	1	1	1	1

ASCII letter A

0	0	0	0	0	0	0	0
50%	50%	50%	50%	50%	50%	50%	50%
50%	50%	50%	50%	50%	50%	50%	50%
1	1	1	1	1	1	1	1

all ASCII chars
in superposition



measuring

0	0	0	0	0	0	0	0
1	1	1	1	1	1	1	1

ASCII char E with
probability 1/256

0 90%	0 10%	0 90%	0 90%	0 90%	0 10%	0 90%	0 10%
10% 1	90% 1	10% 1	10% 1	10% 1	90% 1	10% 1	90% 1



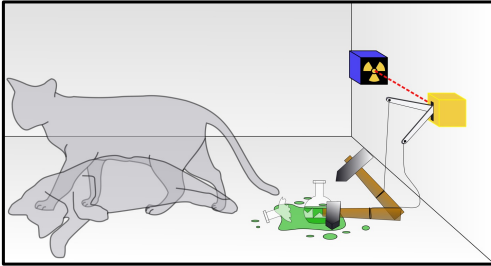
0	0	0	0	0	0	0	0
1	1	1	1	1	1	1	1

all ASCII chars
in superposition
(but probably E)

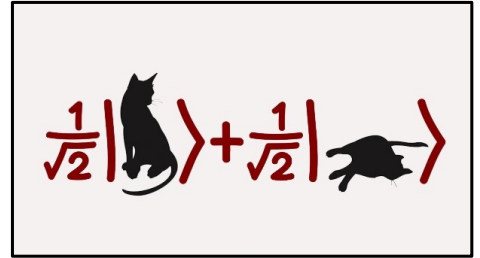
measuring

ASCII char E with
probability 0.43

Schrödinger's Cat



en.wikipedia.org/wiki/Schrödinger%27s_cat

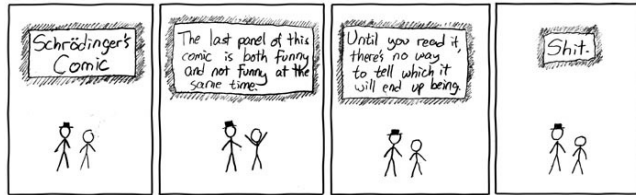


www.motherjones.com/kevin-drum/2018/09/schrodingers-cat-is-alive-one-twelfth-of-the-time/

Schrödinger's Cat



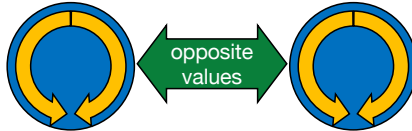
tvtropes.org/pmwiki/pmwiki.php/
UsefulNotes/SchrodingersCat



xkcd.com/45



Entanglement



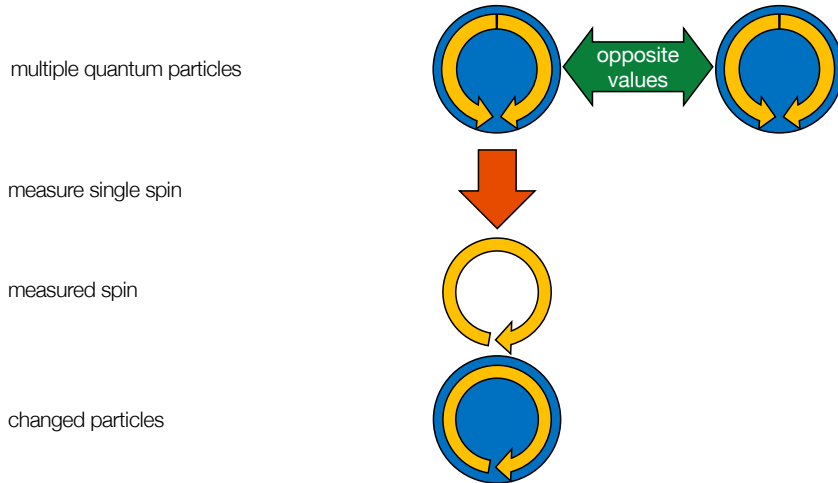
multiple quantum particles

measure single spin

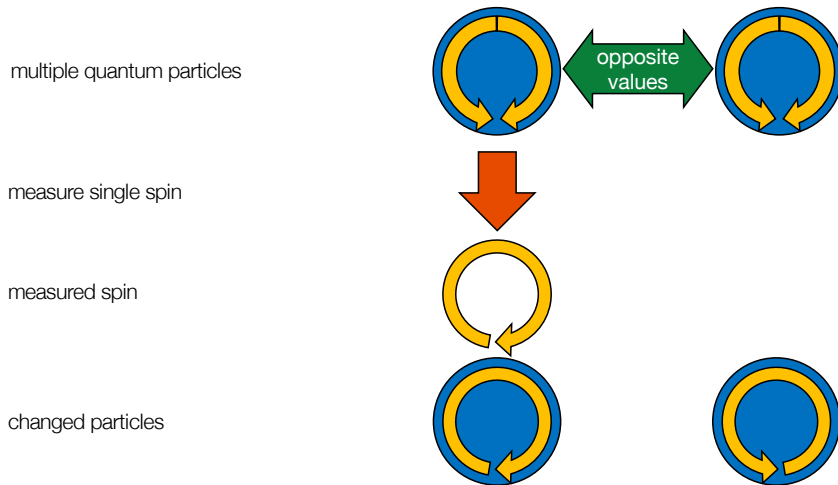
measured spin

changed particles

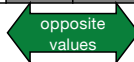
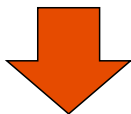
Entanglement



Entanglement



0	0	0	0	0	0	0	0
1	1	1	1	1	1	50% 1	50% 1



0	0	0	0	0	0	0	0
1	1	1	1	1	1	1	1

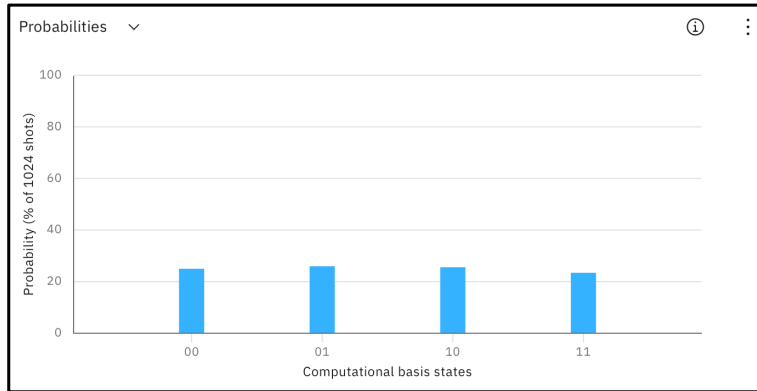
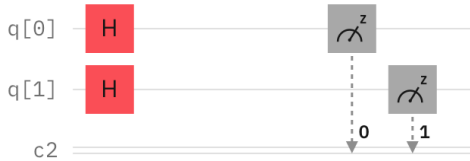
ASCII chars A B at
the same time

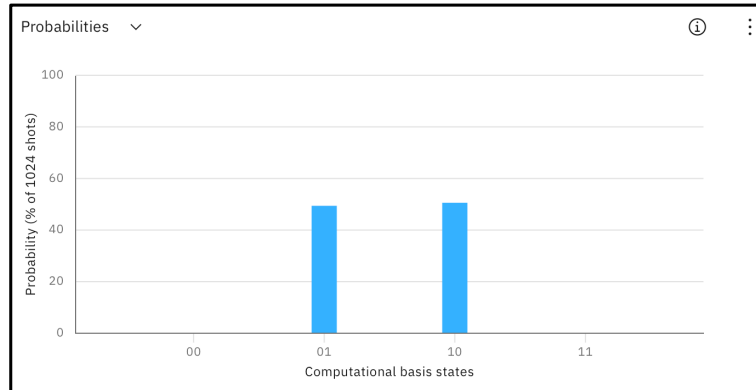
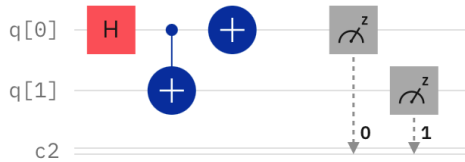
measuring

ASCII char A with
probability 1/2

Let's try

quantum-computing.ibm.com/composer





generated with quantum-computing.ibm.com/composer

Definition 6 (quantum gate). Let $q = \sum_{i=0}^{2^n-1} c_i |i\rangle$ be an n -qubit register. We often write

$$q = \begin{bmatrix} c_{0\dots 0} \\ \vdots \\ c_{1\dots 1} \end{bmatrix}$$

as a shorter notation when the basis $|0\dots 0\rangle, \dots, |1\dots 1\rangle$ is fixed.

An n -ary quantum gate is an operator that works on an n -qubit register q , i.e., (possibly) alters its superposition amplitudes, and thus produces a new n -qubit register q' . An n -ary quantum gate can be given via a matrix $U \in \mathbb{C}^{n \times n}$ so that

$$q' = Uq.$$

Quantum gates commonly implemented on quantum computers are unitary, which — among other things — implies they are reversible.

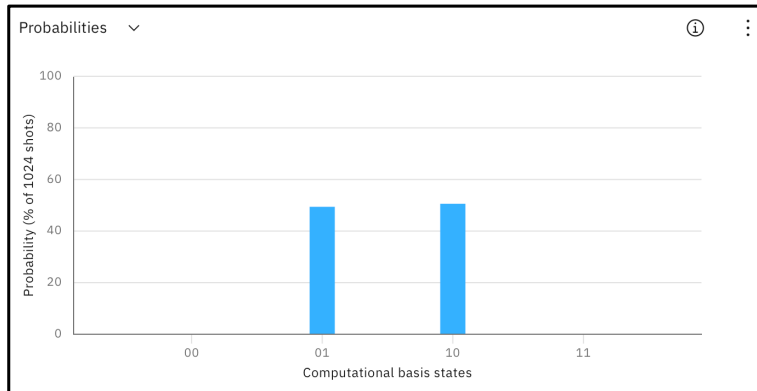
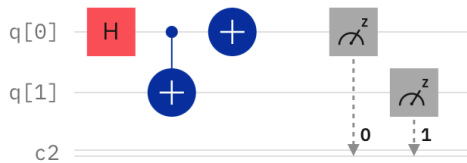
A little excursion:

Why do computers get hot
when they compute?

A little excursion:

Why are most quantum operators
reversible?

Operator	Gate(s)		Matrix
Pauli-X (X)			$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$
Pauli-Y (Y)			$\begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$
Pauli-Z (Z)			$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$
Hadamard (H)			$\frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$
Phase (S, P)			$\begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix}$
$\pi/8$ (T)			$\begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{bmatrix}$
Controlled Not (CNOT, CX)			$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$
Controlled Z (CZ)			$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix}$
SWAP			$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$
Toffoli (CCNOT, CCX, TOFF)			$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$



generated with quantum-computing.ibm.com/composer

Quantum Advantage

One more time!

Theorem 1 (Church-Turing thesis). Any computable function can be computed by a Turing machine.

Theorem 2 (extended Church-Turing thesis). Any computable function can be computed by a Turing machine *with at most polynomial overhead*.

Quantum Advantage?

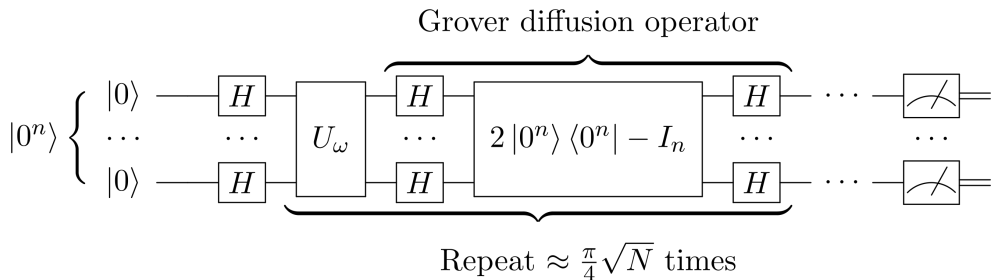
Grover's Search

Problem: Search in unsorted data structure with N elements

Classical Complexity: $\mathcal{O}(N)$

Quantum Complexity: $\mathcal{O}(\sqrt{N})$

en.wikipedia.org/wiki/Grover%27s_algorithm



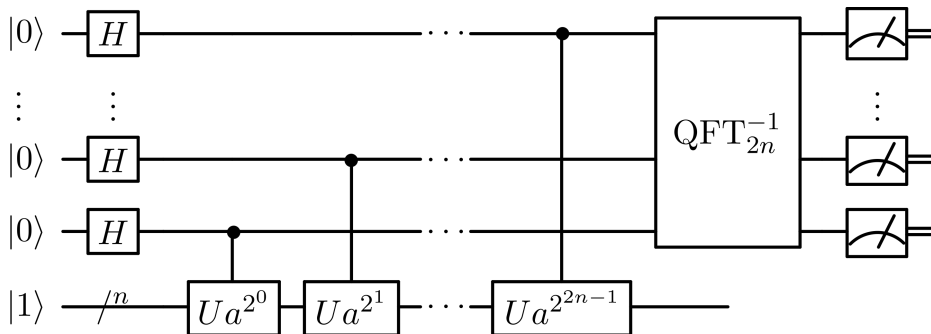
Shor's Algorithm

Problem: Prime factorization of integer N

Classical Complexity: best-known exponential in $\log N$

Quantum Complexity: polynomial in $\log N$

en.wikipedia.org/wiki/Shor%27s_algorithm



HHL Algorithm

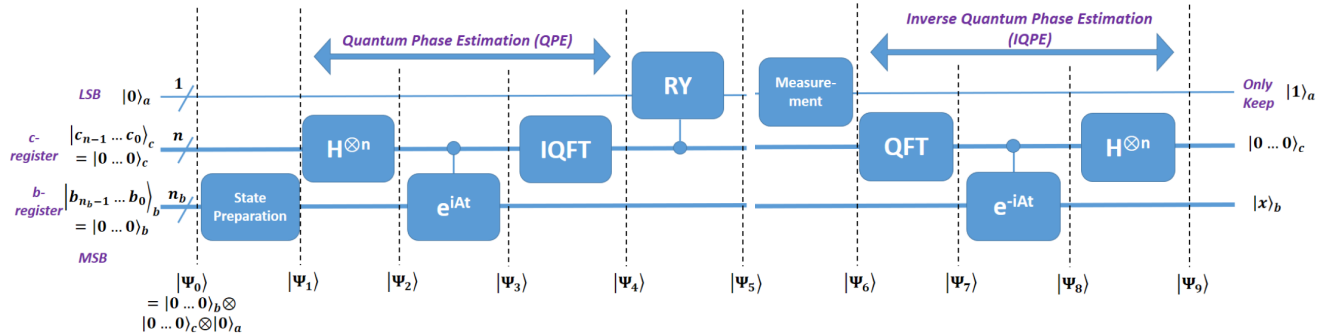
named after
Harrow, Hassidim, Lloyd

Problem: Solving linear systems of N equations with condition number κ

Classical Complexity: $\mathcal{O}(\sqrt{\kappa}N)$ for a scalar value computed from solution

Quantum Complexity: $\mathcal{O}(\kappa^2 \log N)$ for a scalar value computed from solution

adapted from arxiv.org/pdf/2108.09004.pdf



Quantum Annealing

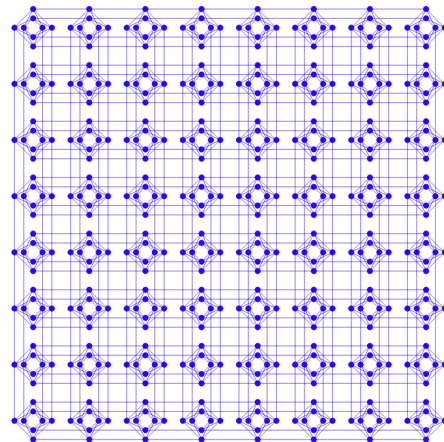
introduced by
Kadowaki, Nishimori

Problem: Solving (quadratic unconstrained binary) optimization problems

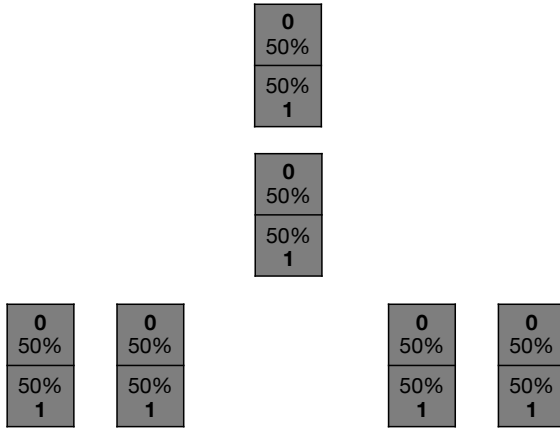
Classical Complexity: best-known exponential for exact solution

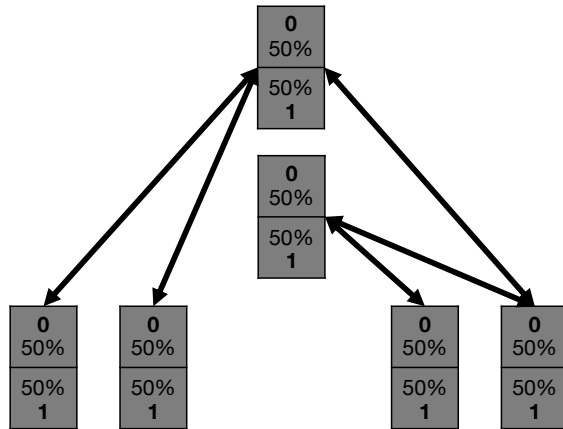
Quantum Complexity: probably exponential as well?

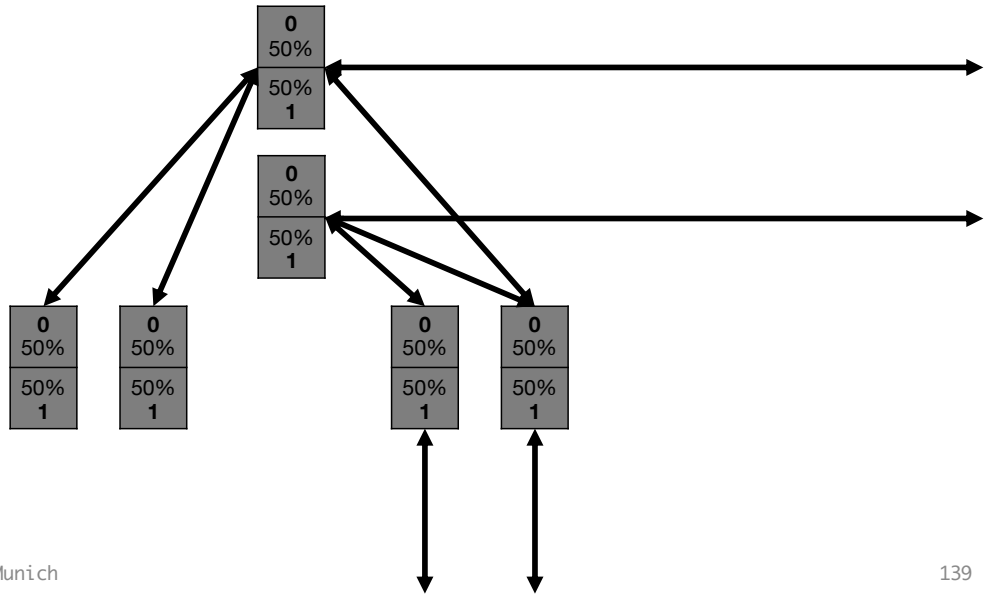
arxiv.org/pdf/1406.2741.pdf

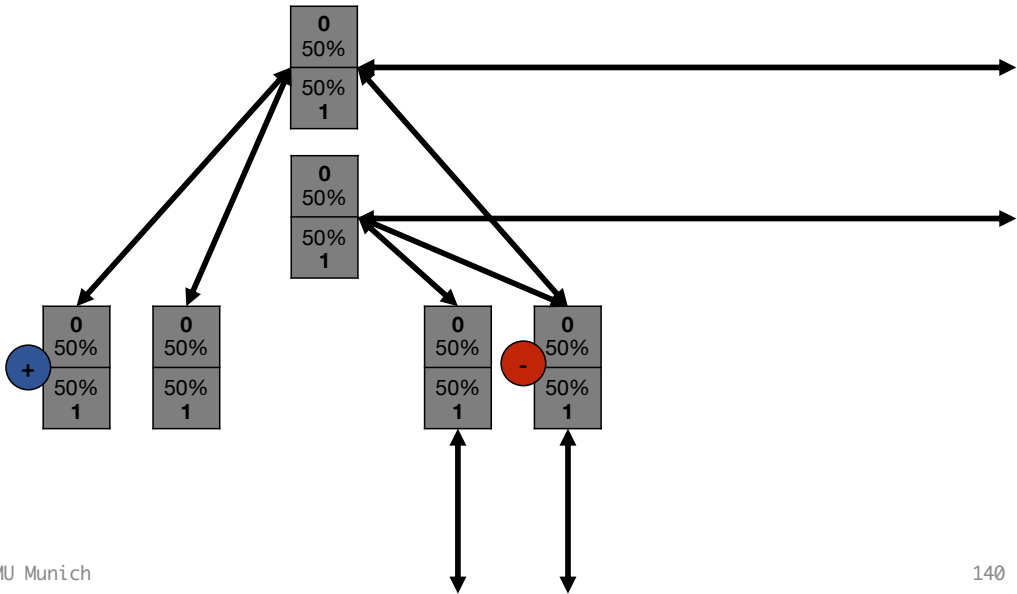


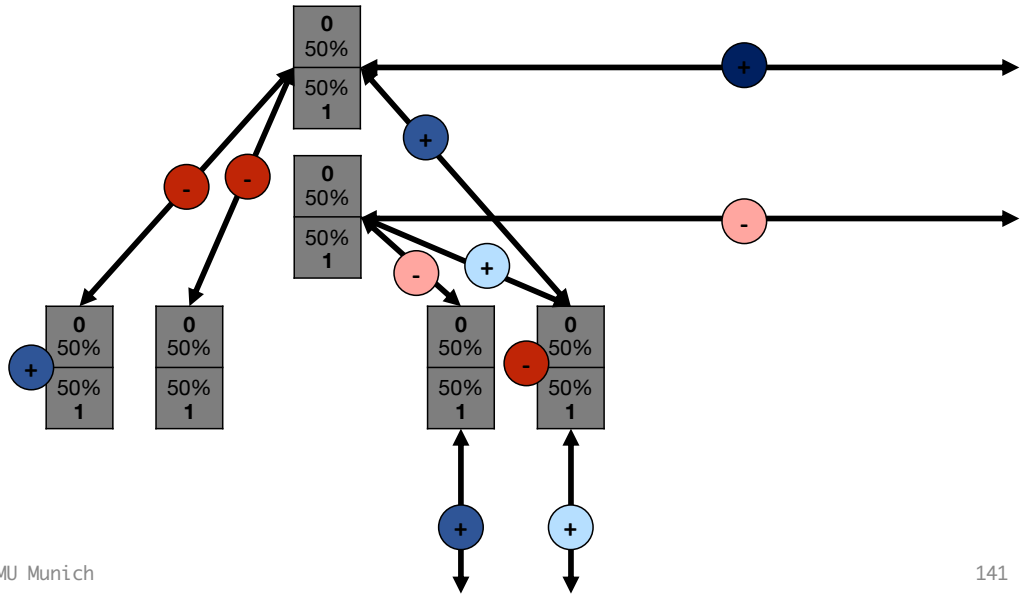
So what is
quantum annealing?





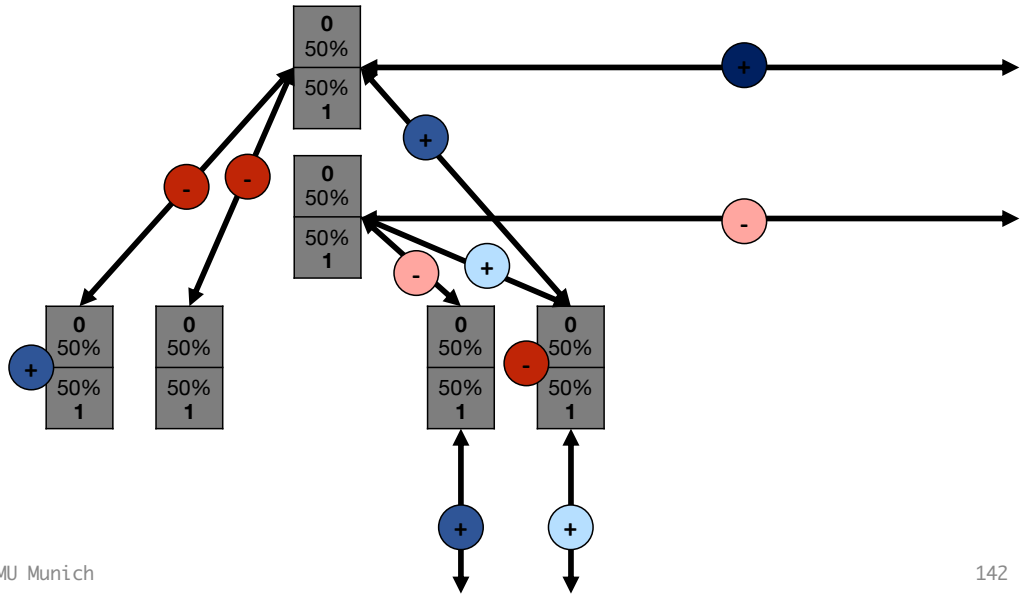




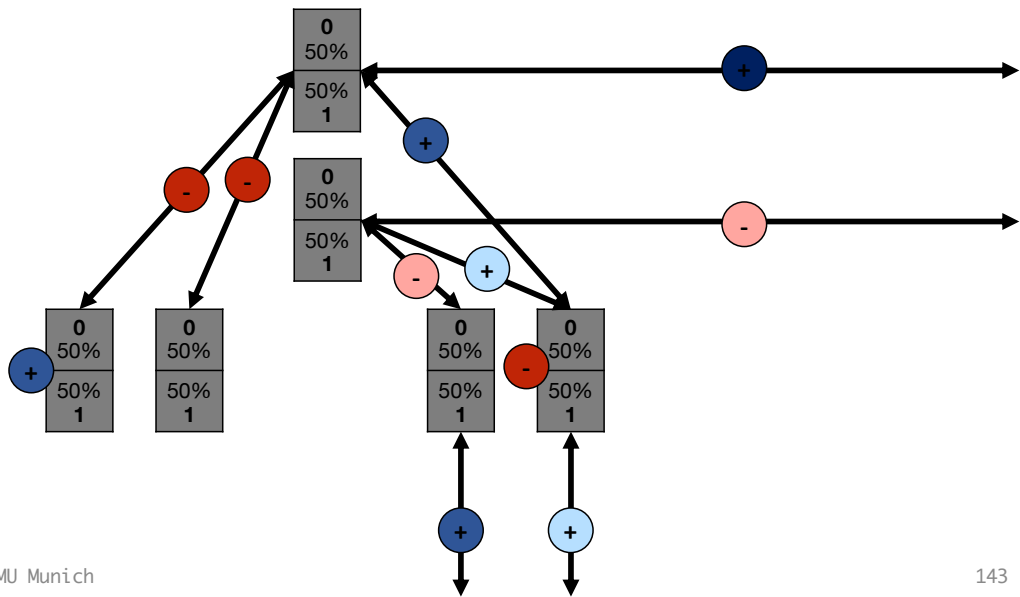


adiabatic quantum computing

add constraints
infinitely slowly
while adding
no energy

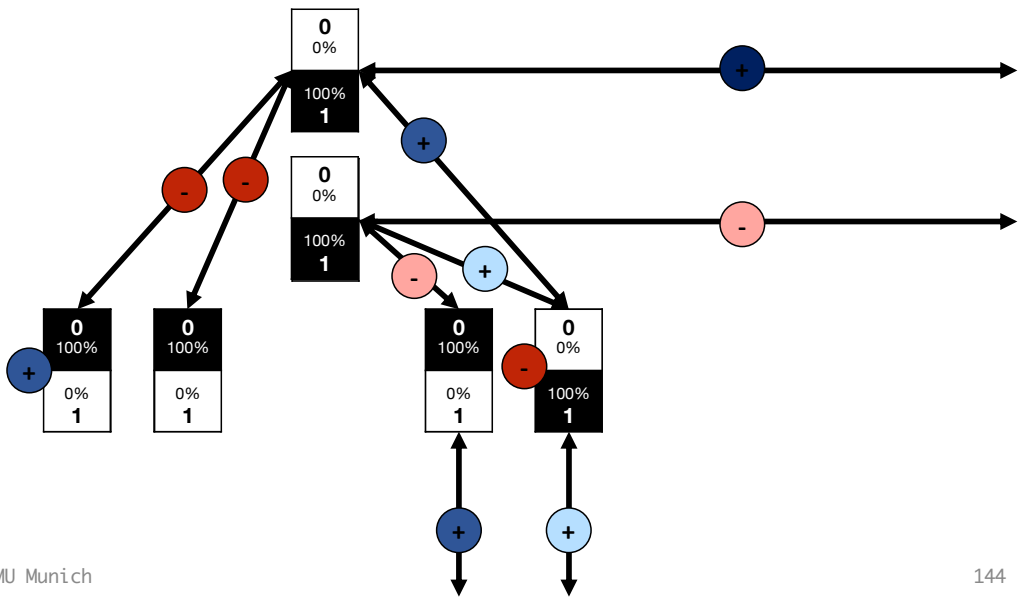


practical quantum annealing





measuring



Definition 7 (Ising model). Let $G = (V, E)$ be a graph with vertices $V = \{v_0, \dots, v_{n-1}\}$ for $n \in \mathbb{N}$ and undirected edges $E \subseteq V \times V$. A spin configuration $x \in \mathcal{X} = (V \rightarrow \{-1, +1\})$ is a mapping of vertices to values $\{-1, +1\}$. W.l.o.g. we assume that x has the form $x(v_i) = \sigma_i$ for any $i \in \{0, \dots, n-1\}$ and a spin vector $\vec{\sigma} \in \{-1, +1\}^n$. The energy of an Ising model is then given via

$$H(\vec{\sigma}) = - \sum_{\substack{i \in \{0, \dots, n-1\} \\ j \in \{0, \dots, n-1\}}} J_{ij} \sigma_i \sigma_j - \sum_{i \in \{0, \dots, n-1\}} h_i \sigma_i$$

where $J \in \mathbb{R}^{n \times n}$ is called the interaction matrix and $\vec{h} \in \mathbb{R}^n$ is called the vector of field strengths. We also assume that for any $i, j \in \{0, \dots, n-1\}$ it holds that $i \geq j \implies J_{ij} = 0$, i.e., J is a strictly upper triangle matrix, and that $(v_i, v_j) \notin E \implies J_{ij} = 0$, i.e., J_{ij} can only have non-zero values where an edge between i and j exists in the substrate graph G .

For an Ising model given by (G, J, h) , there are processes (both artificial and natural) that produce a spin configuration $\vec{\sigma}$ so that $H(\vec{\sigma})$ is minimized.

Definition 8 (QUBO). Let $Q \in \mathbb{R}^{n \times n}$ be an arbitrary matrix called QUBO matrix. The process of finding a vector $\vec{x} \in \mathbb{B}^n$ so to

$$\text{minimize } H(\vec{x}) = x^\top Q x = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} Q_{ij} x_i x_j$$

is called *quadratic unconstrained binary optimization (QUBO)*. W.l.o.g. we can assume that Q is a (non-strictly) upper triangular matrix.

Ising–QUBO Equivalence